Практическое задание №5. Одномерные массивы

Оценка

- На 3: **только на 3**.
- На 4: **только на 4.**
- На 5: **сначала на 4 и затем на 5.**

Данные можно взять из примера (если есть), а можно свои: заполнять случайными числами, вводить с клавиатуры или задавать прямо в коде.

Оценка 3

1. Объявить массив на 12 целых чисел. Пусть это будет температура за год, но подойдут любые числа.
2. Заполнить любым способом.
3. Вывести массив в столбик в формате `a[0] = -5`, `a[1] = 4` и так далее.
4. Посчитать и вывести:
   - сумму всех элементов;
   - среднее арифметическое с двумя знаками после запятой;
   - максимальный и минимальный элемент, а также их индексы.

Оценка 4

Задача про отсчёты сигнала. Комплексное число храним двумя частями: реальной (re) и мнимой (im). Так в системах связи описывают амплитуду и фазу, и именно так упакованы IQ-данные из лекции про массивы.

1. Объявить массив размером N (например, N = 10) для комплексных чисел.
2. Заполнить значения в диапазоне [-32768..32767]: случайными числами, вводом с клавиатуры или прямо в коде.
3. Вывести отсчёты в формате как в примере:

```
s[0] = -1234.0 + j*(-3000.0)
```

4. Вывести энергию сигнала. Формула:

$$E = \sum_{k=0}^{n-1} (re_k^2 + im_k^2)$$

```
(сумма по k от 0 до n-1 от re[k]*re[k] + im[k]*im[k])
```

5. Найти и вывести максимальный элемент среди реальной части и его индекс.

Оценка 5

Выбираем вариант по списку (по модулю 16). Если задание кажется невыполнимым, то можно выбрать +- 2 от текущего.

№	Трек	Вариант
1	A	Качество Wi-Fi по уровню сигнала
2	B	Нормализация записи
3	C	Сглаживание температуры
4	D	Метрики сервера и обещание по скорости
5	A	Детектор движения по Wi-Fi
6	B	Чистка щелчков
7	C	Выбросы датчика
8	D	Частота кодов ответа
9	A	Компаратор с гистерезисом
10	B	Поиск пауз
11	C	Зоны пульса
12	D	Цены и просадка
13	B	Сжатие одинаковых отсчётов
14	D	Плейлист без повторов одного исполнителя
15	C	Дребезг кнопки
16	D	Проверка ключей

Трек A. Связь и сигналы

A1. Качество Wi-Fi по уровню сигнала

Сценарий. Роутер раз в час пишет уровень сигнала клиента в dBm. Надо понять, где связь держалась нормально, а где проваливалась.

Вход. Массив из 24 замеров (сутки по часам):

```
int rssi[24] = {-42, -55, -67, -81, -88, -79, -60, -45, -39, -52, -63, -74, -85, -90, -77, -58, -44, -41, -53, -66, -72, -80, -59, -47};
```

Что вывести.

- Сколько замеров попало в зоны: отличная связь (больше -50), рабочая (от -50 до -70), слабая (меньше -70).
- Средний уровень за сутки.
- Самый длинный провал: серия замеров слабее -70, с номерами начала и конца.

Функции. Подсчёт по зонам, среднее, самая длинная серия ниже порога.

A2. Детектор движения по Wi-Fi

Сценарий. Уровень сигнала между двумя устройствами меняется, когда в комнате кто-то ходит. Так делают простые системы присутствия.

Вход. Массив из 60 замеров, по одному в секунду. Для проверки хватит и 12:

```
int rssi[12] = {-60, -59, -61, -58, -70, -74, -68, -57, -58, -62, -75, -71};
```

Что вывести.

- Список интервалов активности: скачок больше 5 dBm между соседними замерами считается началом, возврат к спокойному уровню - концом.
- Количество интервалов, суммарную длительность в секундах и самый длинный интервал (номер начала и конца).
- Сколько процентов времени было спокойно.

Функции. Разности соседних замеров, поиск интервалов, суммарная длительность, самый длинный интервал.

A3. Компаратор с гистерезисом

Сценарий. Так работают триггеры в осциллографах и входы микроконтроллеров: сигнал с шумом нельзя сравнивать с одним порогом, иначе получается дребезг. Берут два порога.

Вход. Массив из 20 отсчётов, верхний порог 1200, нижний 800:

```
int raw[20] = {120, 300, 900, 1500, 1700, 1600, 900, 700, 850, 1100, 1400, 1600, 1500, 900, 600, 400, 300, 500, 1200, 1500};
```

Что вывести.

- Строку из нулей и единиц: единица, когда сигнал выше верхнего порога, и держится, пока не упадёт ниже нижнего.
- Количество фронтов (переходов с нуля на единицу).
- Длительности импульсов в отсчётах.

Функции. Проход с гистерезисом, подсчёт фронтов, длительности импульсов.

Трек B. Звук

B1. Нормализация записи

Сценарий. Запись получилась тихой. Нужно поднять громкость до предела, но так, чтобы громкие места не срезались.

Вход. Массив из 16 отсчётов, значения как в 16-битном звуке, от -32768 до 32767:

```
int x[16] = {120, -3400, 8000, -12000, 500, -700, 15000, -9000, 300, -150, 6000, -3000, 900, -400, 1100, -250};
```

Что вывести.

- Самый громкий отсчёт по модулю.
- Коэффициент усиления: 32767, делённое на этот максимум. Вывести с двумя знаками после запятой.
- Нормированный массив: каждый отсчёт умножен на коэффициент, округлён до целого.

- Сколько отсчётов было бы срезано, если применить усиление дважды (сравнить с 32767).

Функции. Поиск максимума по модулю, масштабирование, подсчёт срезанных отсчётов, вывод массива.

B2. Чистка щелчков

Сценарий. В записи слышны щелчки. Это отдельные отсчёты, которые резко выбиваются из своего окружения. Их можно заменить средним соседей.

Вход. Тот же массив, но с парой выбросов внутри:

```
int x[16] = {120, -3400, 8000, -12000, 500, -700, 15000, -9000,
300, -15000, 6000, -3000, 900, -400, 1100, -250};
```

Что вывести.

- Номера отсчётов, которые считаются щелчками, и их значения.
- Массив после замены щелчков на среднее двух соседей.
- Сколько отсчётов исправлено.

Функции. Проверка, похож ли отсчёт на своих соседей, замена, подсчёт исправлений.

B3. Поиск пауз

Сценарий. Запись лекции надо нарезать на куски по тишине. Значит сначала нужно найти паузы.

Вход. Массив из 24 значений громкости, одно значение это 100 мс. Порог тишины 200, минимальная пауза 3 значения (то есть 0.3 секунды):

```
int amp[24] = {800, 1200, 90, 40, 30, 1500, 1400, 60, 50, 45, 900,
1300, 1100, 70, 1500, 1600, 55, 40, 1400, 1200, 80, 700, 1100,
1300};
```

Что вывести.

- Список пауз: время начала в секундах и длительность в секундах.
- Количество пауз и самую длинную.
- Общее время тишины и его долю от всей записи.

Функции. Поиск пауз, перевод в секунды, поиск самой длинной, подсчёт доли.

B4. Сжатие одинаковых отсчётов

Сценарий. В записи есть длинные участки одного уровня: тишина, ровный фон. Их выгодно хранить парой: значение и сколько раз оно повторилось подряд.

Вход. Массив из 20 значений:

```
int x[20] = {3, 3, 3, 3, 7, 7, 9, 9, 9, 9, 9, 9, 2, 4, 4, 4, 8,
8, 8, 8};
```

Что вывести.

- Таблицу серий длиннее двух: значение и длина.
- Самую длинную серию.

- Во сколько раз сжалось: исходное количество чисел, делённое на количество пар.
- Обратную распаковку: собрать массив из пар заново, вывести его и написать, совпал ли он с исходным.

Функции. Печать серий, распаковка обратно, подсчёт коэффициента.

Трек С. Датчики и микроконтроллеры

С1. Сглаживание температуры

Сценарий. Датчик на улице пишет температуру, но показания немного дёргаются. Нужно сгладить ряд и посмотреть, был ли устойчивый рост.

Вход. Массив за 14 дней:

```
int t[14] = {-5, -4, -6, -3, -1, 0, -2, 1, 2, 4, 3, 5, 6, 7};
```

Что вывести.

- Сглаженный ряд: каждое значение это среднее трёх соседей (сам отсчёт и два рядом). Печатать с одним знаком после запятой. На краях массива усредняй по тем соседям, которые есть.
- Самый длинный период роста: подряд идущие дни, когда значение не уменьшалось. Вывести номер первого дня и длину периода.
- Насколько сглаживание убрало разброс: разницу между максимумом и минимумом до и после.

Функции. Сглаживание, поиск периода роста, разброс ряда.

С2. Выбросы датчика

Сценарий. Часть измерений явно бракованная: значение резко отличается от остальных. Такие точки надо найти и заменить, иначе среднее врёт.

Вход. Массив из 20 измерений:

```
int x[20] = {20, 21, 19, 22, 20, 21, 20, 22, 19, 20, 21, 20, 55,
21, 20, 19, 22, 20, 21, 20};
```

Что вывести.

- Среднее значение по массиву.
- Номера измерений, которые отличаются от среднего больше чем на 15.
- Массив после замены этих измерений на среднее двух соседей.
- Среднее значение после исправления, для сравнения.

Функции. Среднее, поиск выбросов, замена, вывод массива.

С3. Зоны пульса

Сценарий. Фитнес-браслет пишет пульс раз в секунду. Спортсмену нужно понять, сколько он был в каждой зоне и не рвал ли нагрузку.

Вход. Массив за 24 секунды:

```
int hr[24] = {85, 92, 98, 105, 118, 130, 142, 148, 151, 147, 150,
160, 165, 158, 143, 138, 120, 110, 104, 96, 90, 88, 150, 152};
```

Что вывести.

- Сколько секунд пульс был в зонах: покой (меньше 100), разминка (100..139), кардио (140..169), пик (170 и выше).
- Самую длинную непрерывную серию в кардио или выше: номер первой и последней секунды, длительность.
- Средний пульс за запись.

Функции. Определение зоны по значению, подсчёт времени по зонам, поиск самой длинной серии, среднее.

C4. Дребезг кнопки

Сценарий. Микроконтроллер читает кнопку каждую миллисекунду. Из-за дребезга за одним нажатием приходят несколько коротких срабатываний. Нужно их убрать.

Вход. Массив из 24 значений, единица значит, что кнопка нажата. Одно значение это 1 мс, минимальное честное нажатие 3 мс:

```
int raw[24] = {0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0};
```

Что вывести.

- Очищенный сигнал: короткие импульсы длиной меньше 3 не считаются нажатием и заменяются нулями.
- Количество нажатий после очистки.

Функции. Очистка сигнала, подсчёт нажатий.

Трек D. Данные, сервисы и быт

D1. Метрики сервера и обещание по скорости

Сценарий. Сервис обещает, что 95 процентов запросов отвечают быстрее 300 мс. В логе есть времена ответа, надо проверить обещание и найти плохие участки.

Вход. Массив из 24 времён ответа в миллисекундах:

```
int ms[24] = {120, 95, 310, 140, 880, 105, 99, 420, 130, 260, 110, 90, 350, 610, 120, 105, 99, 280, 130, 150, 240, 100, 95, 125};
```

Что вывести.

- Самый долгий запрос и его номер.
- Сколько процентов запросов уложились в 300 мс.
- Самый длинный период деградации: серия подряд идущих ответов дольше 300 мс, с номерами начала и конца.

Функции. Поиск максимума, проверка обещания, поиск серии.

D2. Частота кодов ответа

Сценарий. В логе лежат коды ответа сервера. Нужно быстро понять, что там вообще происходило.

Вход. Массив из 24 кодов:

```
int code[24] = {200, 200, 200, 404, 200, 500, 301, 200, 200, 404,
200, 200, 500, 500, 200, 500, 200, 200, 404, 200, 200, 200, 301,
200};
```

Что вывести.

- Таблицу: код и сколько раз он встретился. Строки идут по возрастанию кода.
- Три самых частых кода.
- Гистограмму: для каждого кода столько звёздочек, сколько раз он встретился. Если код встретился один раз, звёздочка всё равно печатается.

Функции. Подсчёт кодов, поиск трёх самых частых, печать гистограммы.

D3. Цены и просадка

Сценарий. Есть цены за 15 дней. Надо оценить, насколько сильно проседала цена и как долго росла.

Вход. Массив из 15 цен:

```
int price[15] = {100, 102, 105, 103, 98, 96, 99, 104, 110, 108,
112, 118, 115, 111, 117};
```

Что вывести.

- Минимум и максимум цены и их дни.
- Максимальную просадку: самое сильное падение от достигнутого максимума. Вывести величину и дни начала и конца.
- Самый длинный период роста: подряд идущие дни, когда цена не падала.

Функции. Поиск минимума и максимума, просадка, период роста.

D4. Плейлист без повторов одного исполнителя

Сценарий. Музыкальные сервисы перемешивают плейлист так, чтобы подряд не шли две песни одного исполнителя. Сделаем то же самое.

Вход. Массив песен из 12 штук и массив исполнителей того же размера. В первом массиве номера песен, во втором номера исполнителей:

```
int song[12]   = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
int artist[12] = {7, 7, 3, 5, 7, 5, 5, 3, 7, 3, 5, 3};
```

Что вывести.

- Исходный порядок песен.
- Перемешанный порядок: тасование Фишера-Йетса (идём с конца и меняем текущий элемент со случайным из начала).
- Сколько соседей одного исполнителя было до перемешивания и сколько осталось после.
- Если исполнителей меньше двух, порядок без соседей построить нельзя: программа должна написать об этом сообщение, а не зацикливаться.

Функции. Тасование, проверка соседей, вывод плейлиста.

D5. Проверка ключей

Сценарий. Система выдаёт кандидатов в ID. Простые числа считаются стойкими и допустимыми, составные хрупкими: у них легко найти делитель. Плюс есть гипотеза Гольдбаха: любое чётное число больше двух раскладывается на сумму двух простых.

Вход. Массив из 12 кандидатов:

```
int id[12] = {17, 24, 91, 97, 100, 121, 13, 60, 77, 88, 101, 39};
```

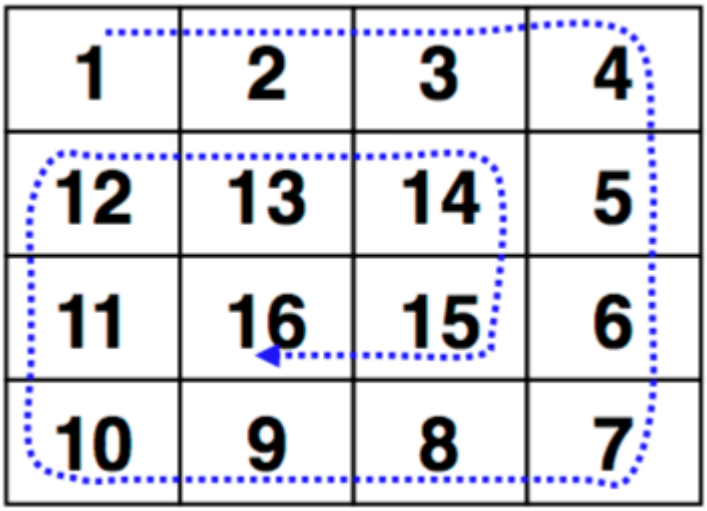
Что вывести.

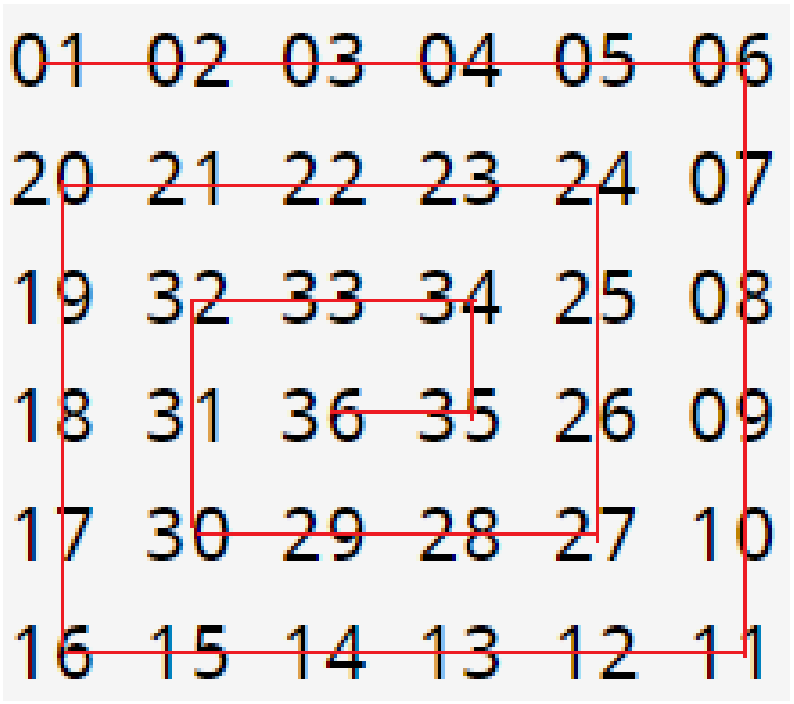
- Для каждого кандидата: простое он или составной. У составного вывести первый найденный делитель.
- Для каждого чётного кандидата не меньше четырёх: разложение на два простых числа, например `24 = 5 + 19`.
- Сколько среди кандидатов простых.

Функции. Проверка простоты, поиск первого делителя, разложение по Гольдбаху.

Звездочка* (+балл)

Спираль (улитка). Построить квадрат N на N и заполнить его числами от 1 до N*N по спирали: числа идут по внешнему кругу и закручиваются внутрь. N вводится с клавиатуры. Примеры для N = 4 и N = 6 есть в задании. Тут нужны два вложенных цикла и аккуратная работа с границами.




01 02 03 04 05 06
20 21 22 23 24 07
19 32 33 34 25 08
18 31 36 35 26 09
17 30 29 28 27 10
16 15 14 13 12 11